

Hardware-In-The-Loop for Manufacturing Automation Control: Current Status and Identified Needs

Fangming Gu, William S. Harrison, Dawn M. Tilbury, Chengyin Yuan

Abstract—In the past simulation and prototyping coexisted as disjointed iterative steps with limited benefits. As technology has advanced and processes have become more complex, the traditional isolated simulation and prototyping approaches can't satisfy dynamic, global competition requirements. Because of this there is a need for a mixture of simulation and physical implementation, called Hardware-in-the-Loop (HIL). This paper will explore the most active fields and research achievements within HIL including component compatibility testing, controller verification, and software. This assessment is used to highlight regions within HIL research that would benefit manufacturing automation control.

I. INTRODUCTION

As design technology has advanced, so too has simulation and emulation to verify new ideas and technologies. From this advancement emerges Hardware-In-The-Loop (HIL). HIL is the bridge between traditionally isolated simulation/emulation and physical prototyping. Before HIL, simulation and prototyping coexisted as disjointed iterative steps. In industry, simulation is considered to be purely a software environment governed by a mathematical model with no real aspects, while emulation is a computer model that mimics the actual system. The simulation/emulation quality highly depends on model accuracy. In addition, the model may not exist, making it impossible to conduct simulation/emulation in some cases. Even with a model, a highly complex control system can not be fully validated until it is tested against a 100 percent accurate model. Prototyping, though capable of producing a realistic model and providing more accurate outcome, results in longer development time, higher engineering cost, less efficiency, and if damaged cannot be reused. HIL efficiently combines these approaches to address current industrial needs. It provides the scalability and cost benefits that come with simulation/emulation while maintaining physical realism at a high level. As described in [19] there are several things that motivate the use of HIL: intense pressure to shorten the development cycle, enhanced safety requirements which mandate thorough testing of control systems, increased need to prevent failures at all parts of a product life cycle, reduced cost, and increased reuse of already proven available systems and components.

This paper surveys the most active fields in HIL, and addresses the the key characteristics of ideal HIL solutions

for manufacturing automation. The paper is organized as follows: the background and problem statement are defined in sections 2 and 3, while the literature search is presented in section 4, 5, and 6. The summary includes the conclusion and the author's view of HIL for future development.

In this paper, HIL is divided into two categories, HIL Electronics (controllers and implementation platforms) and Physical HIL (components). The HIL electronics section includes controller validation scenarios and open source programming platforms well suited for HIL. The physical HIL section covers approaches for physical apparatus other than controllers. Other possible categorizations of HIL could be training versus testing or distributed versus centralized control, for example.

II. BACKGROUND

HIL was originally used for testing control behavior for real time embedded systems, such as electric device controllers or vehicular control units. A typical HIL setup would include a controller with loaded control code connected to a testing environment as shown in Figure 1. In past years however HIL has expanded to encompass component testing as well.

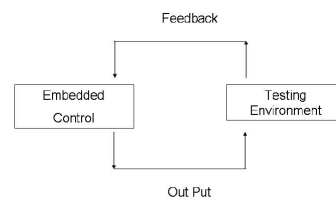


Fig. 1. Development Cycle

The most popular usage of HIL is control validation. Although in some places control validation and verification are used interchangeably, this paper distinguishes between them. Controller verification is the mathematical proving of a system covering all scenarios, therefore verifying the control algorithm. Validation uses actual testing to determine that the controller works; it is usually impossible to test all possible scenarios. Validation will be discussed in this paper. Generally, HIL for control validation includes any controller that is connected to a simulation/emulation for the purpose of control logic validation. Another popular implementation of HIL entails testing a physical component or system by connecting it to a simulation/emulation environment. Engine testing with a dynamometer would be one example of this kind of HIL. The third and probably the least recognized

This work was supported in part by the National Science Foundation ERC for Reconfigurable Manufacturing Systems, grant number EEC 95-92125.

F. Gu and C. Yuan are with General Motors, Warren, MI

W. S. Harrison and D. M. Tilbury are with the University of Michigan, Department of Mechanical Engineering, Ann Arbor, MI 48109-2125. Corresponding author: wsh@umich.edu

TABLE I
IDEAL HIL FOR MANUFACTURING AUTOMATION

1. Any level virtual-real combination
2. Seamless part handling and data exchange
3. "Plug and play" (modular characteristics)
4. Controller and components unaware of state of other parts
5. Real-time event-driven data handling
6. Scalable and hierarchical
7. Standardized signal coupling and interface

application of HIL is system compatibility testing. It focuses on emulating the connectivity of the actual system, which can extend the benefits of all the other uses. The details of each usage will be explored further in succeeding sections.

HIL is important because of its benefits over pure simulation/emulation and prototyping. HIL is a mixture of actual process pieces interacting with simulation/emulation. By replacing the simulation model with real hardware components, the experiment is closer to the real world scenario. The increased reliability can make process planning and prediction more accurate.

HIL is not widely used in manufacturing automation partly because of the misunderstandings surrounding it such as: it is considered to be mostly for embedded control, it appears to add additional steps to the development process, and it will increase cost. In this paper, we will show that HIL is not just for embedded control. It can be used for anything that fits into a large complex system. It does require hardware investment and additional development steps prior to prototyping, which at first may seem cost extra. However the total cost and development time can be reduced significantly.

III. PROBLEM STATEMENT

Most HIL applications are mostly virtual with the real parts only representing a small portion of the complete system. The opposite (mostly real) is much less common. Outside of controller testing, manufacturing automation research that focuses on the mix of simulation/emulation and real equipment is similarly unknown. Although HIL is heavily influenced by embedded control testing and validation, applicable work to manufacturing automation does exist. A method that takes full advantage of HIL benefits should have the key characteristics outlined in Table I.

One of the most challenging aspects of developing a manufacturing process is the characterization and prediction of how the process will behave with a complex array of subsystems. Current HIL research focuses on controllers. It does not cover how a real CNC (component) machine might interact with a virtual robot (component) for example. HIL could be used to explore the complexity that arises from large multi component systems. Part handling across the virtual and real domains is another challenge that must be addressed when developing HIL for manufacturing automation.

IV. PHYSICAL HILS: TESTING COMPONENTS

Physical HIL (PHIL) includes components and physical apparatuses (other than controllers). PHIL is beneficial in performance prediction and compatibility testing. Though

this field is not widely researched, there are some applications that have been extensively studied. Engine testing using dynamometers for example is a popular way to test engine characteristics [2]. Within manufacturing automation it is very rare however to have a PHIL process where peer entities (components of equal functional level and importance) of both real and virtual status interact. Two examples of this lesser studied field are distributed Robot Systems (DRS) and Space Robot Emulation. These two deal with the interaction between real and virtual in a hybrid capacity.

A. Distributed Robot Systems

A mobile robot based Distributed Robot System (DRS) consists of a group of decentralized autonomous robots. The decentralized aspect means that there is no master robot/CPU, centralized/shared memory, or synchronized clock in the system. The robots must work together to complete specific tasks [27]. PHIL provides a great platform for testing DRS because of its virtual scalability. If the system must contain a large number of robots, it is impractical to implement such a system. This is because of limiting factors like cost, space, and time. PHIL also makes development a safer process. Without PHIL, the robot testing has to be done with real obstacles in a real environment. The system must then interact with human operators. This can be dangerous because sensory information is often flawed when only simulation results are available [12].

Kormoriya *et al.* first proposed to place a real robot in a virtual environment in 1990 [12], to bridge the gap between full simulation and complete physical implementation. Their setup consisted of a robot controlled in free space while a host computer contains the environment. The host computer knows the location of the robot at all times and can simulate ultrasonic and laser range sensory information.

In 1999 Dixon *et al.* developed a Real And Virtual Environment (RAVE) for multiple robot systems. RAVE allows real robots to interact with virtual robots in a virtual environment. This is accomplished by putting virtual sensors along with real sensors into the system with real and virtual robots. Virtual sensors can be attached to real and virtual robots. This scenario gives rise to hybrid sensors which are real and virtual sensors fused together.

This concept is further explored by Hu *et al.*, who developed "Robot-In-the-Loop" (RIL) [8]. They created a robotic convoy system using the RIL concept [7]. A group of robots were put together to form a synchronized sequential line. Because they are using a RIL methodology some of the robots in the line are real and some are virtual.

B. Special Purpose Dextrous Manipulator

In the aerospace domain, there is a world wide initiative to build the International Space Station (ISS). One important portion of the ISS is the Special Purpose Dextrous Manipulator (SPDM) [1, 3, 21, 22]. The SPDM focuses on orbital replacement units or scientific payloads.

In most cases a space robot can be verified through simulation; however this is challenging for the SPDM because

reliable contact force models do not exist. Contact force calculations are difficult because of the complex nature of the frictional contact between the payload and the work site. These difficulties are further compounded because most space robots cannot support their own weight.

A PHIL architecture was developed in which one robot can be modeled in the simulation and a rigid real robot is used to simulate the contact dynamics. The real robot is made as close as possible to the simulated SPDM, but for explained reasons is not an exact match. This concept was used successfully to verify parts of the SPDM design.

C. PHIL Summary

The selected projects highlight the ongoing research in PHIL. In general, most of the above mentioned work could be considered robot-in-the-loop (RIL). It bridges traditional simulation and physical system execution seamlessly while enhancing safety. The work also illustrates how well this method fits large scale cooperative robot systems and separates control from sensor operator interface. These strengths coincide specifically with some of the qualities listed in Table I: (1) achieving any level of virtual and real testing, (3) plug and play for individual robots, (4) the robots are unaware of other components, and (6) scalable architecture. Meanwhile, some shortcomings of current RIL research are identified, such as not covering part handling. This is understandable within the context of DRS, but points to a larger area in RIL that should be studied.

There are other applications of PHIL, however they are sparsely scattered through the engineering sciences. Simulation followed by complete prototype implementation for physical parts is a design tradition. In order to change this trend, benchmarking studies should be done that compare efficiency and process design with and without PHIL. The conducted and ongoing PHIL research has some direct impact on manufacturing automation. It can speed up robot integration, improve safety, and test large-scale systems. While manufacturing automation systems are becoming more complex, most components must be in place to test a hierarchical structured approach. With PHIL, the ramp up time could be much shorter and the systems will be much better. The authors expect that more attention should be given to compatibility testing, which becomes a more important topic for manufacturing automation control development.

V. HIL ELECTRONICS

A. HIL for Controllers

The use of HIL for control validation is quite far reaching and has been used in applications as diverse as mass spring dampers [26] and control engineering education [5]. This widespread use is because HIL can reduce time and cost for controller development.

This section is divided into two sub-sections both of which are based on controller testing: manufacturing controllers (i.e. PLC), and vehicular controllers (i.e. embedded systems). Manufacturing automation controllers and vehicular

controllers are addressed because they are the closest link to manufacturing automation.

1) *HIL for Manufacturing Controllers*: HIL Controller validation in manufacturing is not widely utilized. Sometimes emulation is used for validating small pieces of the code. This is done via loading it on to a vendor dependant programmable logic controllers (PLC) connected to a small mockup of the system. Sometimes control code validation is done with a soft PLC (PC based controller). This can technically be considered HIL, but it is vendor specific, lacking in hardware and not easily applicable to other parts of the system. Although HIL for manufacturing controllers is not done extensively, there has been research in Soft Commissioning; Schluderman *et al.* [25] tests industrial control software by connecting a PLC to a discrete simulation/emulation software tool (Arena).

Manufacturing PLC development could benefit significantly from more HIL usage. Most of the current research is vendor specific. An interesting topic would be seamless integration of multi vendor products (hardware and software) through HIL to achieve scalable logic validation.

2) *HIL for Vehicular Controllers*: The automotive industry has been using HIL for vehicular controller validation for quite a while. Since vehicle control is becoming less strictly mechanical and more electromechanical, the use of embedded controllers will continue to increase. Generally HIL is used for testing vehicle control systems, where the real controller interacts with a virtual vehicle. Traditionally, a control unit could only be tested by simulation or mule build. Mule build testing is costly and time consuming because controller testing must wait until the other vehicle related parts and subsystems are finished. Test by implementation is also costly because many different required scenarios may not be feasible without massive redesign, or extensive damage. Simulation or emulation on the other hand may use inaccurate models and might not be able to address key issues.

As cars in general become more and more computerized and complex due to emission constraints and energy restrictions, so too has the powertrain. The late 90s witnessed a sharp increase in HIL applications [19]. Already in 1998, the Ford Research Laboratory was able to assess the issues associated with using HIL for powertrain control design [23].

The Anti-Lock Braking System (ABS), and Traction Control Systems (TCS) are some other appropriate areas for HIL. Testing these controllers would definitely put an expensive prototype at risk. Lee *et al.* [14] were able to complete an analysis of an ABS/TCS system along with a COTS electronic control unit using a PC based HIL approach [13].

The conventional breaking systems will soon be replaced by Brake-By-Wire (BBW) systems. Both Heo *et al.* [6] and Choi *et al.* [4] use HIL to test their BBW systems.

Steer-By-Wire (SBW) is a part of the drive by wire system that is not yet widely implemented by car manufacturers. Jang *et al.* [10], Park *et al.* [20], and Huh *et al.* [9] all use HIL to develop or test their control systems. In all three cases

they used HIL to test their respective controller and control logic before actually installing them in the prototype.

One of the newest additions to the automotive world is the Hybrid Electric Vehicle (HEV). The HEV has more complex control systems than the typical gasoline vehicle and thus can benefit greatly from the use of HIL. Ramaswamy *et al.* used HIL for their vehicle system controller at first and then found that they could use HIL to test a large portion of the entire Electronic Control Unit [24]. Wang *et al.* used HIL to develop the powertrain controller [28].

3) *HIL Testing Controllers Summary:* HIL for controller validation is widely researched. The developed methods do a good job with signal acquisition and testing analysis. Some weaknesses include: it sometimes requires complete and accurate vehicle simulation models combined with real components to achieve high-accuracy simulation/validation, validation requires customized setup and can not be expanded, it is limited to small applications, and can not cover large scale complex systems well, and setups are mostly simulation/emulation with small real parts. Though these shortcomings take away from the applicability to automation control, it does contribute some advancements, these include: (3) modular component dynamic testing, (5) real-time data exchange and handling, and efficient control algorithm verification. Unfortunately not all of these advances coincide with the ideal HIL qualities listed in Table I, so there is still room for improvement.

B. HIL Implementation Platform: Real Time Virtual Test Bed

Many software applications have been developed for HIL, such as RT-Lab, RTDS, DSPACE and HyperSim [18]. These are all proprietary software tools that are not open to the general research community for improvement. Nearly all of them are well documented and described by their respective vendors and will not be discussed here. There does however, exist an open source software environment that is specifically useful for HIL called Real Time Virtual Test Bed (RTVTB).

The Real Time Virtual Test Bed (RTVTB) originally was developed as the Virtual Test Bed (VTB) by the University of South Carolina [15]. VTB was designed for prototyping complex electronic systems. The motivation came from systems that have a reasonably varied set of processes and components. When dealing with such a diverse system the simulation should be equally as diverse, meaning that multiple simulation tools that are best suited for individual subsystems should be used together to achieve optimal performance. The VTB achieves this goal by doing distributed computation which makes it ideal for HIL applications.

The Real Time Virtual Test Bed (RTVB) was made to be real time by using a real-time Application Interface (RTAI) [16]. The RTAI allows the RTVTB to run with a hard real-time system clock, enabling its applicability to all other real time applications. The RTVTB is a multi-platform, multi-solver, hard real-time prototyping environment that delivers an efficient solution for HIL testing. It is also a very low cost environment and completely comprised of open-source software and reasonably priced off the shelf hardware.

RTVTB has been applied to power electronic controls [17], motor drive systems [18], a fuel-cell battery-charging station [11], and has proven its capability as an accurate HIL testing tool for varied processes.

RTVTB makes a good case for being a low cost alternative to proprietary software used for HIL. Its applicability to manufacturing automation is somewhat questionable because of the lack of examples using RTVTB for manufacturing. It also has not been implemented on a large-scale complex system, therefore inferring how it would work in such a scenario is difficult. Its distributed approach seems to be well suited for manufacturing automation.

HIL software in general including both commercial and open source can decrease development time. One of its limitations may come in its close marriage to simulation as opposed to emulation. Out of the seven ideal qualities covered in Table I, RTVTB could possibly be very good at having a “plug and play” capability. In general, it would benefit the research and industrial community to have more effort put into developing open source alternatives and investigating wider applications, thereby diversifying the influence of HIL software.

VI. SUMMARY AND FUTURE WORK

As stated above there are various research efforts in HIL for the use of controller validation, and there are several COTS tools available for supporting this approach. PHIL, on the other hand, highlights a great disparity in HIL. It is good for testing component performance and compatibility. Analyzing a component’s individual behaviors and interactions would have great use in manufacturing automation. Hybrid systems, like the RIL or RAVE, focus component compatibility testing within HIL. A manufacturing process which addresses this would include real and virtual machines interacting in a seamless unaware fashion where true process behavior can be analyzed.

Most of the identified research has virtual and real entities that interact with another in only one way; other types of interactions are excluded. In general entities can interact in three ways: spatial physical (collision), direct information connection (digital or analog), and physical exchange (part or device handed from one entity to another). Most HIL systems only cover direct information connection, which means that subsystems are connected via wiring and communicate with analog or digital signals. The RIL and RAVE systems on the other hand are exceptions. They use spatial physical and direct information connections.

None of the literature surveyed addresses physical exchange, i.e. passing parts inherent to one environment into another (from virtual to real and vice versa). This would include passing a virtual object which is inherently virtual to a real environment. Physical/virtual exchange is not well studied because the typical HIL setup is not capable. In most cases a setup only has two distinct regions of real and virtual. The two regions communicate through direct information connection and nothing else. The authors propose that HIL should have the capability to handle an infinite number of

distinct regions. Each virtual region can be bordered by two real ones and vice versa. Qualities 2 and 3 from Table I make this possible.

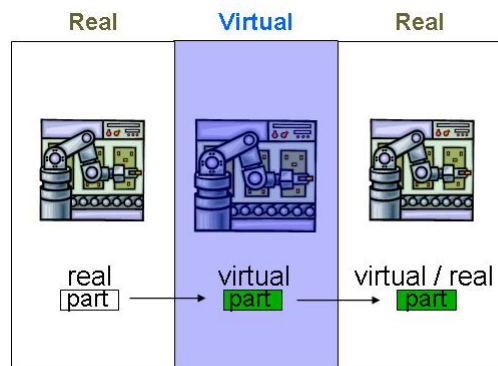


Fig. 2. Part transition through real and virtual regions

It would be of particular interest in manufacturing automation design and testing to study how parts pass between real and virtual regions. This interaction highlights an important unresolved challenge of PHIL compatibility testing. A virtual object in a virtual environment and a real object in a real environment are trivial scenarios. What happens when a virtual part is passed into the real environment? An example of this transition is shown in Figure 2. When the part is initially in the real region it exists as a normal part, however after it passes through the virtual region and returns to the real, it must exist as a mix of both real and virtual. PHIL for compatibility testing requires that these transitions between environments be handled correctly. If some regions of a manufacturing process are virtual and the rest are real, a part must be able to transition between the two domains seamlessly.

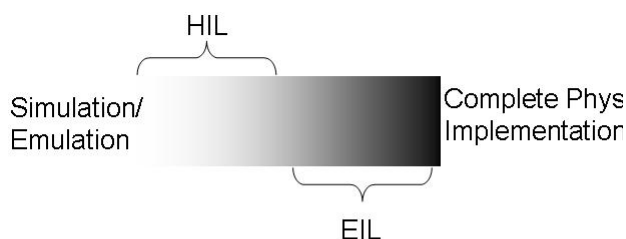


Fig. 3. Development progression from virtual to physical implementation

The real/virtual testing disparity highlights the difference between a HIL process that is mostly simulation and one that is mostly real and physically present. The literature predominantly considers situations that are mostly virtual. A good example would be the steer by wire system, where the entire car is modeled while the steering system is real. The reverse is rarely done, and doing both in the same process development is even less prevalent. The reason for this most likely lies in the cost effectiveness of a virtual system. The heterogeneous mix (hybrid) of process regions (virtual/real)

requires a new approach in perspective and concept. This new approach takes the focus away from simulation and shifts it to process interaction and compatibility. Simulation is still an important but different issue. Even assuming perfect simulation a hybrid process (virtual and real) could still not be equivalent to the actual process.

Creating an equivalent process should be the focus of any hybrid system. An equivalent process means: if all simulations are assumed to be perfect then the process is the same as its complete physical counterpart. Though a complete equivalent process does not mean the simulations are perfect, achieving perfect equivalence should be the goal. This concept separates simulation and hybrid process fidelity. Equivalence focuses more on the large scale process connectivity and compatibility bringing more inertia to the HIL development cycle. A development cycle should start from completely virtual (simulation/emulation), to partially virtual (HIL), to mostly real, (as seen in Figure 3) and then completely real (physical prototype) as a natural design progression. As this progression takes place and a system becomes more real and less virtual it could then be considered emulation in the loop (EIL). Only then will HIL be truly utilized as a development tool. With this approach the entire automation process, development, testing, control and final monitoring can all be covered by HIL at different levels, and the information should be seamlessly exchanged between layers and regions. This will finally provide support to entire organization planning and management.

REFERENCES

- [1] F. Aghili and J. Piedboeuf, *Emulation of robots interacting with environment*, IEEE/ASME Transactions on Mechatronics **11** (2006), 35–46.
- [2] B. Bunker, M. Franchek, and B. Thomason, *Robust multivariable control of an engine-dynamometer system*, IEEE Transactions on Control Systems Technology **5** (1997).
- [3] J. Carufel, E. Martin, and J. Piedboeuf, *Control strategies for hardware-in-the-loop simulation of flexiblespace robots*, IEEE Proceedings Control Theory and Applications, 2000.
- [4] S. Choi, M. Cho, and N. Wereley, *Wheel-slip control of a passenger vehicle using an electrorheological valve pressure modulator*, Proceedings of the Institution of Mechanical Engineers, 2006.
- [5] W. Grega, *Hardware-in-the-loop simulation and its application in controlled education*, 29th ASEE/IEEE Frontiers in Education Conference, 1999.
- [6] S. Heo, K. Park, and H. Ahn, *Design and implementation of hardware-in-the-loop simulator for EHB systems*, SICE Annual Conference, 2003.
- [7] X. Hu, N. Ganapathy, and B. Zeigler, *Robots in the loop: supporting an incremental simulation-based design process*, IEEE International Conference on Systems, Man and Cybernetics (2005).
- [8] X. Hu and B. Zeigler, *A simulation-based virtual environment to study cooperative robotic systems*, Integrated Computer-Aided Engineering **12** (2005), 353–367.
- [9] K. Huh and J. Kim, *Active steering control based on the estimated tire forces*, Journal of Dynamic Systems, Measurement, and Control **123** (2001), 505–511.
- [10] S. Jang, T. Park, and C. Han, *A control of vehicle using steer-by-wire system with hardware-in-the-loop-simulation system*, IEEE/ASME International Conference on Advanced Intelligent Mechatronics, 2003.
- [11] Z. Jiang, R. Leonard, R. Dougal, H. Figueroa, and A. Monti, *Processor-in-the-loop simulation, real-time hardware-in-the-loop testing, and hardware validation of a digitally-controlled, fuel-cell powered battery-charging station*, IEEE 35th Annual Power Electronics Specialists Conference, 2004.

- [12] K. Komoriya and K. Tani, *Utilization of the virtual environment system for autonomous control of mobile robots*, Proceedings of the IEEE International Workshop on Intelligent Motion Control, 1990, p. 20.
- [13] J. Lee and M. Suh, *Hardware-in-the loop simulator for ABS/TCS*, Control Applications, 1999. Proceedings of the 1999 IEEE International Conference on, 1999.
- [14] W. Lee, M. Yoon, and M. Sunwoo, *A cost- and time-effective hardware-in-the-loop simulation platform for automotive engine control systems*, Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering, 2002.
- [15] T. Lovett, A. Monti, E. Santi, and R. Dougal, *A multilanguage environment for interactive simulation and development controls for power electronics*, IEEE 32nd Annual Power Electronics Specialists Conference, 2001.
- [16] B. Lu, W. McKay, S. Lentijo, A. Monti, X. Wu, and R. Dougal, *The real time extension of the virtual test bed*, Proceedings of the Huntsville Simulation Conference, 2003.
- [17] B. Lu, A. Monti, and R. Dougal, *Real-time hardware-in-the-loop testing during design of power electronics controls*, The 29th Annual Conference of the IEEE Industrial Electronics Society, 2003.
- [18] B. Lu, X. Wu, and A. Monti, *Implementation of a low-cost real-time virtual test bed for hardware-in-the-loop testing*, 32nd Annual Conference of IEEE Industrial Electronics Society, 2005.
- [19] D. Maclay, *Simulation gets into the loop*, IEE Review **43** (1997), 109–112.
- [20] T. Park, C. Han, and S. Lee, *Development of the electronic control unit for the rack-actuating steer-by-wire using the hardware-in-the-loop simulation system*, Mechatronics **15** (2005), 899–918.
- [21] J. Piedboeuf, M. Doyon, R. LArcheveque, E. Martin, and M. Lambert, *Simulation environments for space robot design and verification*, Tech. report, ASTRA, 2000.
- [22] J. Piedboeuf, F., M. Aghili, Y. Doyon, E. Gonthier, Martin, and W.H. Zhu, *Emulation of space robot through hardware-in-the-loop simulation*, Proceeding of the 6 th International Symposium on Artificial Intelligence and Robotics & Automation in Space, 2001.
- [23] B. Powell, N. Sureshbabu, and M. Dunn K. Bailey, *Hardware-in-the-loop vehicle and powertrain analysis and control design issues*, Proceedings of the American Control Conference, 1998.
- [24] D. Ramaswamy, R. McGee, S. Sivashankar, A. Deshpande, J. Allen, K. Rzemien, and W. Stuart, *A case study in hardware-in-the-loop testing: Development of an ECU for a hybrid electric vehicle*, SAE Technical Paper Series **01** (2004).
- [25] H. Schludermann, T. Kerchmair, and M. Vorderwinkler, *Soft-commissioning: Hardware-in-the-loop-based verification of controller software*, Proceedings of the 2000 Winter Simulation Conference, 2000.
- [26] D. Shetty, R. Kolk, J. Kondo, and C. Campana, *A new approach to mechatronic systems design using hardware in the loop simulation*, IEEE/ASME International Conference on Advanced Intelligent Mechatronics Proceedings (2001).
- [27] J. Wang, *Methodology and design principles for a generic simulation platform distributed robotic system experimentation and development*, IEEE International Conference on Computational Cybernetics and Simulation, 1997.
- [28] J. Wang, Y. Luo, Y. Cheng, K. Li, and X. Lian, *Hardware-in-the-loop simulator of HEV powertrain controller*, China Mechanical Engineering **16** (2005), 1478–1480.